

Отчёт по лабораторной работе №7

Дискретно-событийное моделирование: модель
М/М/с и модель Росса

true

Цель работы

Освоить дискретно-событийный подход к моделированию систем массового обслуживания на языке Julia с использованием пакетов ConcurrentSim и ResumableFunctions. Реализовать модель М/М/с и модель Росса (система с резервом и ремонтом), провести параметрические исследования и сравнить результаты симуляций с аналитическими оценками. Оформить скрипты в литературном стиле с помощью Literate.jl и Quarto.

Задание

- Создать рабочий каталог и инициализировать проект DrWatson для лабораторной работы.
- Установить пакеты: ConcurrentSim, ResumableFunctions, Distributions, StableRNGs, DataFrames, Plots, CSV, Statistics, DrWatson.
- Реализовать базовую модель М/М/с и проверить её работу на тестовом прогоне.
- Перенести модель М/М/с в структуру DrWatson, добавить графики распределения времени ожидания, обслуживания, динамики среднего ожидания и scatter-plot.
- Реализовать модель Росса с мониторингом очереди на ремонт и динамики числа работающих машин.
- Провести параметрическое исследование модели М/М/с для 5 конфигураций; сравнить с аналитической формулой Эрланга.
- Провести параметрическое исследование модели Росса для 7 конфигураций с разным числом запасных машин и ремонтников.

- Преобразовать все скрипты в литературный стиль (файлы *_literate.jl).
- Сгенерировать из литературных скриптов чистый код, Jupyter Notebook и документацию Quarto.
- Выполнить все четыре Jupyter Notebook и убедиться в воспроизводимости результатов.
- Интегрировать сгенерированную Quarto-документацию в отчёт через директивы `{{< include >}}`.

Теоретическое введение

Дискретно-событийное моделирование

Дискретно-событийное моделирование (Discrete-Event Simulation, DES) — подход, при котором состояние системы изменяется только в дискретные моменты времени при наступлении событий. Основные компоненты: событие (мгновенное изменение состояния), очередь (накопитель заявок), обслуживающий прибор (канал, сервер) и генератор входящего потока. В Julia реализация DES строится на пакете ConcurrentSim совместно с корутинами ResumableFunctions.

Модель М/М/с (классификация Кендалла)

По классификации Кендалла, запись М/М/с означает: пуассоновский входящий поток с интенсивностью λ , экспоненциальное время обслуживания с интенсивностью μ и c параллельных каналов. Загрузка системы: $\rho = \lambda / (c \cdot \mu)$. Условие стационарности: $\rho < 1$.

Вероятность ожидания (формула Эрланга С):

$$P(\text{ожидание}) = \frac{\frac{(c\rho)^c}{c!(1-\rho)} P_0}{\frac{(c\rho)^c}{c!(1-\rho)} P_0 + 1}$$

где P_0 — вероятность простоя всех каналов.

Формулы Литтла для связи числа заявок в системе и времени ожидания: $L = \lambda W$, $L_q = \lambda W_q$.

Модель Росса

Система с конечной популяцией: N работающих машин, S запасных в резерве, R ремонтников. При отказе рабочей

машины она немедленно заменяется резервной, сломанная отправляется в ремонт. Если резерва нет — система падает (crash). После ремонта машина возвращается в пул резерва. Цель — оценить среднее время $E[T]$ до падения системы.

Table 1: Пакеты Julia, используемые в лабораторной работе {#tbl-packages}

Пакет	Назначение
ConcurrentSim	Дискретно-событийное моделирование (DES)
ResumableFunctions	Корутины для процессов симуляции
Distributions	Экспоненциальные и другие распределения
StableRNGs	Воспроизводимые генераторы случайных чисел
DataFrames	Хранение и анализ результатов экспериментов
Plots	Визуализация результатов
CSV	Сохранение результатов в CSV-файлы
DrWatson	Организация проекта и воспроизводимость
Literate	Генерация кода, notebooks и документации

Выполнение лабораторной работы

Настройка рабочего окружения

Перешёл в каталог `labs/lab07` и запустил Julia 1.12.6 (релиз от 2026-04-09). В REPL выполнил `using DrWatson` и `initialize_project("project")` — создана стандартная структура каталогов DrWatson (рис. @fig-001).

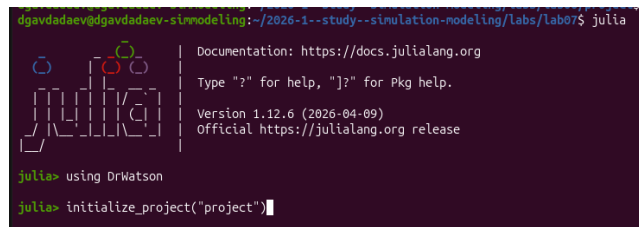


Figure 1: Запуск Julia 1.12.6 и инициализация проекта DrWatson

Перешёл в каталог `project/` и запустил Julia с флагом `--project=.`, выполнил `using Pkg, Pkg.status()`. Установлены и активированы все зависимости: CSV v0.10.16, ConcurrentSim v1.5.1, DataFrames v1.8.2, Distributions v0.25.126, DrWatson v2.19.1, Plots v1.41.6, ResumableFunctions v1.0.6, StableRNGs v1.0.4 (рис. @fig-002)).

```
dgavdadaev@dgavdadaev-simmodeling:~/2026-1--study--simulation-modeling/labs/lab07$ cd project/
dgavdadaev@dgavdadaev-simmodeling:~/2026-1--study--simulation-modeling/labs/lab07/project$ julia --project-.

Documentation: https://docs.julialang.org
Type "?" for help, "j?" for Pkg help.
Version 1.12.6 (2026-04-09)
Official https://julialang.org release

julia> using Pkg

julia> Pkg.status()
Status: ~/2026-1--study--simulation-modeling/labs/lab07/project/Project.toml
[336ed66f] CSV v0.10.16
[6ed1e86c] ConcurrentSim v1.5.1
[a93c6f08] DataFrames v1.8.2
[31c24e18] Distributions v0.25.126
[62482b9d] DrWatson v2.19.1
[91a5bcd0] Plots v1.41.6
[c5292f4c] ResumableFunctions v1.0.6
[868ef19b] StableRNGs v1.0.4
```

Figure 2: Статус проекта: установленные пакеты с версиями

Базовая реализация модели М/М/с

Модуль MMcQueue.jl

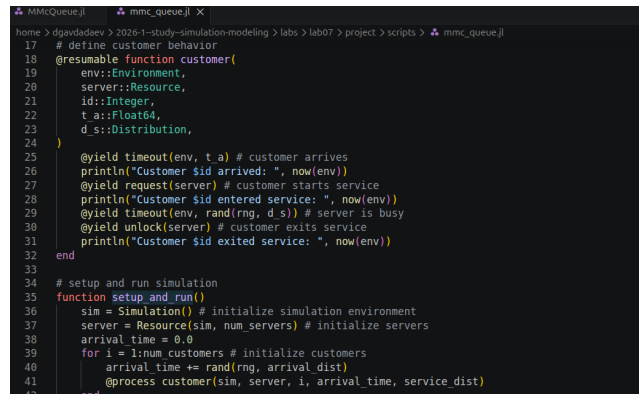
В VS Code создан файл src/MMcQueue.jl — модуль с корутиной customer, которая реализует поведение заявки: прибытие, ожидание освобождения сервера (@yield request(server)), обслуживание и освобождение. Генератор случайных чисел зафиксирован как StableRNG(123) ([рис. @fig-003]).

```
MMcQueue.jl X
home > dgavdadaev > 2026-1--study--simulation-modeling > labs > lab07 > project > src > MMcQueue.jl
1 module MMcQueue
2
3 using ConcurrentSim
4 using ResumableFunctions
5 using Distributions
6 using StableRNGs
7 using DataFrames
8
9 export run_mmc_simulation
10
11 const rng = StableRNG(123)
12
13 @resumable function customer(
14     env::Environment,
15     server::Resource,
16     id::Int,
17     arrival_time::Float64,
18     service_dist,
19     log_data,
20 )
21
22     @yield timeout(env, arrival_time)
23
24     arrival = now(env)
25
26     @yield request(server)
27
28     service_start = now(env)
29
30     wait_time = service_start - arrival
```

Figure 3: Модуль MMcQueue.jl: @resumable function customer с @yield request

Скрипт mmc_queue.jl (первая версия)

В scripts/mmc_queue.jl написан скрипт с функцией setup_and_run(): создаётся Simulation(), сервер Resource(sim, num_servers), запускаются процессы для каждой заявки через @process customer(...). Код содержит @resumable function customer с печатью времён прибытия, начала и конца обслуживания ([рис. @fig-005]).



```
17 # define customer behavior
18 @resumable function customer(
19     env::Environment,
20     server::Resource,
21     id::Integer,
22     t_a::Float64,
23     d_s::Distribution,
24 )
25     @yield timeout(env, t_a) # customer arrives
26     println("Customer $id arrived: ", now(env))
27     @yield request(server) # customer starts service
28     println("Customer $id entered service: ", now(env))
29     @yield timeout(env, rand(rng, d_s)) # server is busy
30     @yield unlock(server) # customer exits service
31     println("Customer $id exited service: ", now(env))
32 end
33
34 # setup and run simulation
35 function setup_and_run()
36     sim = Simulation() # initialize simulation environment
37     server = Resource(sim, num_servers) # initialize servers
38     arrival_time = 0.0
39     for i = 1:num_customers # initialize customers
40         arrival_time += rand(rng, arrival_dist)
41         @process customer(sim, server, i, arrival_time, service_dist)
42     end
43 end
```

Figure 4: Скрипт mmc_queue.jl: setup_and_run() с Resource и @process

Запустил скрипт командой `julia --project=. scripts/mmc_queue.jl`. В терминале появился подробный лог: Customer 1 arrived: 0.14518451436852475, Customer 1 entered service: 0.14518451436852475, Customer 3 entered service: 2.2009855201... и т.д. — заявки 4 и 5 ждали, пока освободился канал ([рис. @fig-004]).

Базовая реализация модели Росса

Скрипт ross.jl (первая версия)

В VS Code создан скрипт scripts/ross.jl с параметрами: RUNS = 5, N = 10, S = 3, SEED = 150, LAMBDA = 100, MU = 1. Корутина machine реализует цикл: работа → отказ → получение резервной машины из Store → ремонт → возврат в резерв ([рис. @fig-007]).

Запустил скрипт командой `julia --project=. scripts/ross.jl`. Система симулировала 5 прогонов: время падения составило 12715.7, 37335.5, 30844.6, 1601.3, 824.1 часов. Среднее время до падения системы — 16664.25 часов ([рис. @fig-006]).

```

dgavdadaev@gavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab07/project$ code src/mmc_queue.jl
dgavdadaev@gavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab07/project$ code scripts/mmc_queue.jl
dgavdadaev@gavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab07/project$ cd ..
dgavdadaev@gavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab07$ cd project/
dgavdadaev@gavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab07/project$ julia --project=. scripts/mmc_queue.jl
Customer 1 arrived: 0.14518451436852475
Customer 1 entered service: 0.14518451436852475
Customer 2 arrived: 0.5941831542983594
Customer 2 entered service: 0.5941831542983594
Customer 3 arrived: 1.5498640267819874
Customer 4 arrived: 1.62427969253121217
Customer 5 arrived: 1.691108070989648
Customer 1 exited service: 2.208965528126681
Customer 3 entered service: 2.208965528126681
Customer 6 arrived: 2.2889839524289217
Customer 3 exited service: 3.5822128399442174
Customer 4 entered service: 3.5822128399442174
Customer 7 arrived: 4.377828221628456
Customer 8 arrived: 5.16494276788882
Customer 2 exited service: 5.988722829377648
Customer 5 entered service: 5.988722829377648
Customer 9 arrived: 7.09999441963388785
Customer 10 arrived: 7.828998220943469
Customer 5 exited service: 9.634196437883254
Customer 6 entered service: 9.634196437883254
Customer 4 exited service: 9.678688398447817
Customer 7 entered service: 9.678688398447817
Customer 7 exited service: 15.066978111688014
Customer 8 entered service: 15.066978111688014
Customer 8 exited service: 16.65548432659554
Customer 9 entered service: 16.65548432659554
Customer 6 exited service: 17.481833154878328
Customer 10 entered service: 17.481833154878328
Customer 9 exited service: 17.58868352135993
Customer 10 exited service: 18.698264775288885

```

Figure 5: Лог выполнения mmc_queue.jl: прибытия и обслуживание заявок

```

MMcQueue.jl  mmc_queue.jl  ross.jl  x
home > dgavdadaev > 2026-1--study--simulation-modeling > labs > lab07 > project > scripts > ross.jl
1  using ResumableFunctions
2  using ConcurrentSim
3
4  using Distributions
5  using Random
6  using StableRNGs
7
8  const RUNS = 5
9  const N = 10
10 const S = 3
11 const SEED = 150
12 const LAMBDA = 100
13 const MU = 1
14
15 const rng = StableRNG(42) # setting a random seed for reproducibility
16 const F = Exponential(LAMBDA)
17 const G = Exponential(MU)
18
19 @resumable function machine(
20     env::Environment,
21     repair_facility::Resource,
22     spares::Store{Process},
23 )
24     while true
25         try
26             @yield timeout(env, Inf)
27         catch
28         end
29         @yield timeout(env, rand(rng, F))
30         get_spare = take!(spares)
31         @yield get_spare | timeout(env)
32         if state(get_spare) != ConcurrentSim.idle
33             @yield interrupt(value(get_spare))

```

Figure 6: Скрипт ross.jl: параметры и @resumable function machine

```

dgavdadaev@gavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab07/project$ code scripts/ross.jl
dgavdadaev@gavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab07/project$ julia --project=. scripts/ross.jl
At time 12715.718224958666: No more spares!
At time 37333.335678959807: No more spares!
At time 38844.62667837361: No more spares!
At time 1681.2524911974856: No more spares!
At time 824.1848788405848: No more spares!
Average crash time: 16664.247588264883

```

Figure 7: Запуск ross.jl: 5 прогонов, среднее время краха — 16664.25

Модели в структуре DrWatson

mmc_queue.jl в DrWatson

Переписал scripts/mmc_queue.jl: добавил using DrWatson, @quickactivate "project". Параметры: num_customers = 1000, num_servers = 2, $\mu = 0.5$, $\lambda = 0.9$. Логирование перенесено в DataFrame с полями id, arrival, start_service, finish, waiting, service ([рис. @fig-009]).

```
mmc_queue.jl X ross.jl •
home > dgaiddaev > 2026-1--study--simulation-modeling > labs > lab07 > project > scripts > mmc_queue.jl
1 using DrWatson
2 @quickactivate "project"
3 using StableRNGs
4 using Distributions
5 using ConcurrentSim
6 using ResumableFunctions
7 using DataFrames
8 using Statistics
9 using CSV
10 using Plots
11
12 rng = StableRNG(123)
13
14 num_customers = 1000
15 num_servers = 2
16
17 mu = 0.5
18 lam = 0.9
19
20 arrival_dist = Exponential(1 / lam)
21 service_dist = Exponential(1 / mu)
22
23 log_df = DataFrame{
24     id = Int[],
25     arrival = Float64[],
26     start_service = Float64[],
27     finish = Float64[],
28     waiting = Float64[],
29     service = Float64[],
30 }
31
32 @resumable function customer(
33     env::Environment,
```

Figure 8: mmc_queue.jl в DrWatson: num_customers=1000, num_servers=2, $\lambda=0.9$, $\mu=0.5$

Запустил: среднее время ожидания — 5.900882801930325, среднее время обслуживания — 1.9636150889433126. Результаты сохранены в data/ и plots/ ([рис. @fig-008]).

```
dgaiddaev@dgaiddaev-simulation-modeling:~/2026-1--study--simulation-modeling/labs/lab06/project$ cd ../lab07
dgaiddaev@dgaiddaev-simulation-modeling:~/2026-1--study--simulation-modeling/labs/lab07$ cd project/
dgaiddaev@dgaiddaev-simulation-modeling:~/2026-1--study--simulation-modeling/labs/lab07/project$ julia --project=. scripts/mmc_queue.jl
Среднее время ожидания: 5.900882801930325
Среднее время обслуживания: 1.9636150889433126
Could not create decoration from factory! Running with no decorations.
Результаты сохранены в data/ и plots/
dgaiddaev@dgaiddaev-simulation-modeling:~/2026-1--study--simulation-modeling/labs/lab07/project$
```

Figure 9: Результат mmc_queue.jl DrWatson: среднее ожидание 5.90, обслуживание 1.96

ross.jl в DrWatson

Переписал scripts/ross.jl: RUNS = 10, machine_counts = [5, 10, 20, 30], S = 3, repairers = 2, LAMBDA = 100.0, MU = 1.0. Добавил функцию analytical_estimate(N, S, λ , μ) и мониторинг: repair_queue_log (длина очереди при каждом

отказе), `working_log` (динамика числа рабочих машин) ([рис. @fig-010]).

```

1  using DrWatson
2  @quickactivate "project"
3  using ResumableFunctions
4  using ConcurrentSim
5  using Distributions
6  using StableRNGs
7  using DataFrames
8  using Statistics
9  using CSV
10 using Plots
11
12 RUNS = 10
13 machine_counts = [5, 10, 20, 30]
14 S = 3
15 repairers = 2
16 LAMBDA = 100.0
17 MU = 1.0
18
19 function analytical_estimate(N, S,  $\bar{N}$ ,  $\bar{U}$ )
20      $\bar{N}$ _total = N /  $\bar{N}$ 
21     reserve_factor = S + 1
22     return reserve_factor / max( $\bar{N}$ _total -  $\bar{U}$ , 1e-6)
23 end
24
25 @resumable function machine(
26     env::Environment,
27     repair_facility::Resource,
28     spares::Store{Process},
29     repair_queue_log,
30     working_log,
31     rng,
32     F,
33     G,

```

Figure 10: ross.jl в DrWatson: machine_counts=[5,10,20,30], repairers=2, аналитическая оценка

Запустил: получен DataFrame 4×4 с результатами по каждому N. При N = 5 среднее время краша 7.43×10^5 , при N = 10 — 29970.5, при N = 20 — 2658.15, при N = 30 — 476.449. Средняя очередь на ремонт при N = 30 составила 0.0280 (рис. @fig-011).

[illegible]

Figure 11: Результат ross.jl DrWatson: DataFrame 4×4, N=5→7.43e5, N=30→476.45

В файловом менеджере подтвердилось сохранение результатов: в папке data/ появились файлы mmc_results.csv и ross_model_results.csv ([рис. @fig-012]).

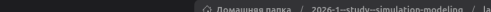


Figure 12: Папка data/ с сохранёнными CSV-файлами результатов

Графики модели М/М/с

На итоговом графике `mmc_plots.png` ([рис. @fig-022]) видны четыре панели. Гистограмма времени ожидания (верхний левый) показывает резкий пик при значениях 0-1 (около 260 заявок) и экспоненциально убывающий хвост до 25-28 единиц — значительная часть заявок обслуживается практически без ожидания. Гистограмма времени обслуживания (верхний правый) имеет аналогичную форму с пиком около 0-0.5 и хвостом до 13 единиц, что соответствует экспоненциальному распределению $\text{Exp}(0.5)$. График динамики среднего ожидания (нижний левый) демонстрирует быстрый рост в начале симуляции до пика около 8.6 на 250-м клиенте, затем постепенное снижение до уровня 6-7 к 1000-й заявке — система выходит на стационарный режим. Scatter-plot «ожидание vs время прибытия» (нижний правый) показывает высокую концентрацию точек у нуля и пачки всплесков ожидания до 25-27 единиц, что отражает эффект скопления заявок.

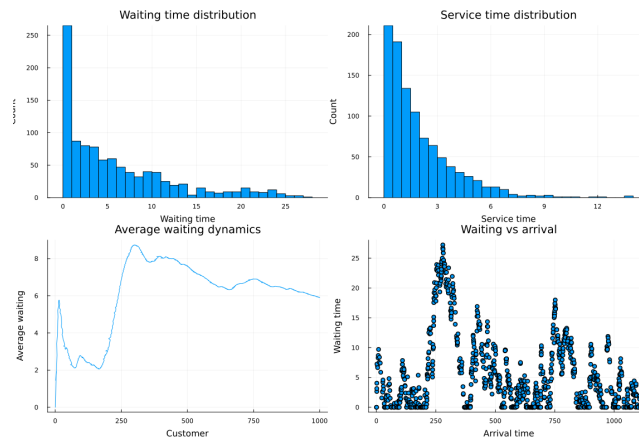


Figure 13: Графики модели М/М/2: гистограммы ожидания и обслуживания, динамика среднего, scatter-plot

Параметрическое исследование М/М/с

Скрипт `mmc_parametrized_literate.jl`

В VS Code создан скрипт `scripts/mmc_parametrized_literate.jl` в литературном стиле. Определены 5 конфигураций: М/М/1 $\rho=1.0$, М/М/2 $\rho=0.5$, М/М/2 $\rho=0.9$, М/М/3 $\rho=0.6$, М/М/4 $\rho=0.75$. Параметры `NUM_CUSTOMERS = 500`, `SEED = 123` ([рис. @fig-014]).

```

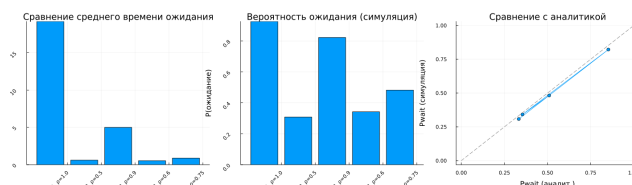
mmc_queue.jl  ross.jl  mmc_parametrized_literate.jl  ross_parametrized_
home > dgavdadaev > 2026-1--study--simulation-modeling > labs > lab07 > project > scripts > mmc_p
1  ## Параметризованные эксперименты: модель M/M/c
2  #
3  # Исследование влияния параметров (λ, μ, c) на характеристики
4  # системы массового обслуживания: время ожидания, вероятность
5  # ожидания, сравнение с аналитическими формулами.
6
7  using DrWatson
8  @quickactivate "project"
9  using StableRNGs
10 using Distributions
11 using ConcurrentSim
12 using ResumableFunctions
13 using DataFrames
14 using Statistics
15 using CSV
16 using Plots
17 using Random
18
19 ## Наборы параметров для экспериментов
20
21 param_sets = [
22     (lam = 0.5, mu = 0.5, c = 1, label = "M/M/1, ρ=1.0"),
23     (lam = 0.5, mu = 0.5, c = 2, label = "M/M/2, ρ=0.5"),
24     (lam = 0.9, mu = 0.5, c = 2, label = "M/M/2, ρ=0.9"),
25     (lam = 0.9, mu = 0.5, c = 3, label = "M/M/3, ρ=0.6"),
26     (lam = 1.5, mu = 0.5, c = 4, label = "M/M/4, ρ=0.75"),
27 ]
28
29 NUM_CUSTOMERS = 500
30 SEED = 123
31
32 ## Формула Эрланга C (вероятность задержки)
33

```

Figure 14: mmc_parametrized_literate.jl: 5 конфигураций param_sets, NUM_CUSTOMERS=500

[illegible]

На графике параметрического исследования ([рис. @fig-024]) видны три панели. Левая — «Сравнение среднего времени ожидания»: при $\rho = 1.0$ столбец М/М/1 достигает ~ 19 , все остальные конфигурации не превышают 5.5 — влияние нестационарности. Средняя панель — «Вероятность ожидания»: М/М/1 имеет $P \approx 0.93$, М/М/2 $\rho=0.9 \approx 0.83$, М/М/4 $\rho=0.75 \approx 0.48$, М/М/2 $\rho=0.5$ и М/М/3 $\rho=0.6 \approx 0.31-0.34$. Правая панель — «Сравнение с аналитикой»: точки симуляции лежат плотно вдоль диагонали, что подтверждает корректность реализации формулы Эрланга.



Параметрическое исследование модели Росса

Скрипт ross_parametrized_literate.jl

11

S=3 R=1, S=5 R=1, S=3 R=2, S=3 R=3, N=5 S=2, N=15 S=5.
 Параметры RUNS = 30, SEED = 42 ([рис. @fig-015]).

```

1 # ## Параметризованные эксперименты: Модель, Расса
2 #
3 # Исследование влияния числа запасных машин (S), количества
4 # ремонтников и числа работающих машин (N) на среднее время
5 # до падения системы.
6
7 using DrWatson
8 @quickactivate "project"
9 using ResumableFunctions
10 using ConcurrentSim
11 using Distributions
12 using StableRNGs
13 using DataFrames
14 using Statistics
15 using CSV
16 using Plots
17
18 # ## Конфигурации экспериментальных прогонов
19
20 param_sets = [
21   (N = 10, S = 1, repairers = 1, lambda = 100.0, mu = 1.0, label = "S=1, R=1"),
22   (N = 10, S = 3, repairers = 1, lambda = 100.0, mu = 1.0, label = "S=3, R=1"),
23   (N = 10, S = 5, repairers = 1, lambda = 100.0, mu = 1.0, label = "S=5, R=1"),
24   (N = 10, S = 3, repairers = 2, lambda = 100.0, mu = 1.0, label = "S=3, R=2"),
25   (N = 10, S = 3, repairers = 3, lambda = 100.0, mu = 1.0, label = "S=3, R=3"),
26   (N = 5, S = 2, repairers = 1, lambda = 100.0, mu = 1.0, label = "N=5, S=2"),
27   (N = 15, S = 5, repairers = 1, lambda = 100.0, mu = 1.0, label = "N=15, S=5"),
28 ]
29
30 RUNS = 30
31 SEED = 42
32
33 # ## Приближённая аналитическая формула

```

Figure 17: ross_parametrized_iterate.jl: 7 конфигураций, RUNS=30, SEED=42

Запустил скрипт. В терминале последовательно вывелись «Запуск: S=1, R=1», «Запуск: S=3, R=1» и т.д. Итоговая таблица 7×11: S=1 R=1 → avg_crash = 103.67; S=3 R=1 → 10700.9; S=5 R=1 → 9.20×10^5 ; S=3 R=2 → 62170.4; S=3 R=3 → 87119.1; N=5 S=2 → 9407.74; N=15 S=5 → 1.42×10^5 ([рис. @fig-016]).

```

Запуск: S=1, R=1
Запуск: S=3, R=1
Запуск: S=5, R=1
Запуск: S=3, R=2
Запуск: S=3, R=3
Запуск: N=5, S=2
Запуск: N=15, S=5

--- Среднее время ---
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
#11 DataFrame
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
row  N    S  repairers  lambda  mu  label  avg_crash  min_crash  max_crash  avg_queue  analytical
----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1    10    1         1    100.0  1.0  S=1, R=1    103.67    1.11774    204.546    0.0          2.0e0
2    10    3         1    100.0  1.0  S=3, R=1  10700.9    270.190    31793.0    0.8187305    4.0e0
3    10    5         1    100.0  1.0  S=5, R=1  9.2049e5    2.49926e6    2.49926e6    0.8113075    6.0e0
4    10    3         2    100.0  1.0  S=3, R=2   62170.4    3034.04    2.24133e5    0.008060198    4.0e0
5    10    3         3    100.0  1.0  S=3, R=3   87119.1    3034.04    2.24133e5    0.0          4.0e0
6     5    2         1    100.0  1.0  N=5, S=2   9407.74    290.951    29640.7    0.0031085    3.0e0
7    15    5         1    100.0  1.0  N=15, S=5  1.4212e5    1.42432e5    14099.1    3.75412e5    0.8267829    6.0e0could not create decoration from factor

Результаты сохранены в data/ в plots/
~/Documents/rossmodeling/2026-1-study-simulation-modeling/lab07/project$
~/Documents/rossmodeling/2026-1-study-simulation-modeling/lab07/project$

```

Figure 18: Результат ross_parametrized_iterate.jl: таблица 7×11 со средними временами краша

На графике базовой модели Расса ([рис. @fig-023]) в верхней левой панели «Average crash time» видно резкое падение времени работы с ростом N: при N = 5 среднее время достигает

$\sim 7 \times 10^5$, при $N = 10$ уже падает до $\sim 3 \times 10^4$, при $N = 20$ — около 2600, при $N = 30$ — всего ~ 480 . Аналитическая оценка (оранжевая линия) даёт значение $\sim 4 \times 10^6$ и заметно превышает симуляционные результаты для всех N — это ожидаемо, поскольку аналитическая формула является грубым верхним приближением. Средняя верхняя панель «Average repair queue» показывает монотонный рост средней очереди на ремонт: от ≈ 0 при $N = 5$ до 0.028 при $N = 30$, что отражает увеличение нагрузки на ремонтников при большем парке машин. Четыре нижние панели «Machines dynamics» демонстрируют различные характеры динамики: при $N = 5$ система колеблется между 5–7 исправными машинами на всём протяжении $\approx 9 \times 10^5$ временных единиц; при $N = 10$ частые отказы создают паттерн 10–12 машин в течение 3×10^4 единиц; при $N = 20$ и $N = 30$ падение системы происходит значительно быстрее.

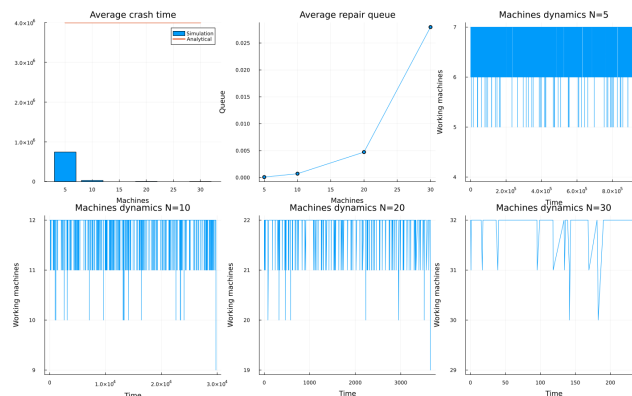


Figure 19: Графики базовой модели Росса: время краша, очередь ремонта, динамика числа машин для $N=5, 10, 20, 30$

На графике параметрического исследования Росса ([рис. @fig-025]) левая панель «Влияние параметров на время работы» показывает доминирующее влияние запасных машин: конфигурация $S=5$, $R=1$ даёт наибольшее время работы — почти 9.2×10^5 , тогда как $S=1$, $R=1$ даёт всего ~ 104 часа. Добавление второго ремонтника при $S=3$ увеличивает время с 10700 до 62170; третий ремонтник даёт 87119. Правая панель «Загрузка ремонтников» (средняя очередь на ремонт) наибольшее значение ≈ 0.026 показывает у конфигурации $N=15$, $S=5$ — при большом числе машин и большом резерве ремонтники заняты постоянно.

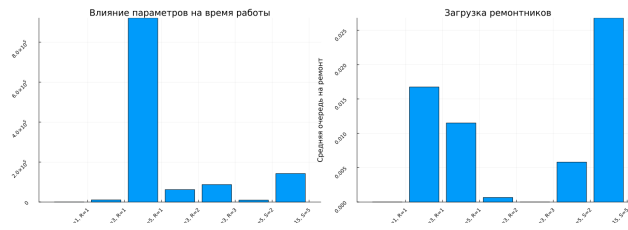


Figure 20: Параметрическое исследование модели Росса: влияние S и R на время до краша, загрузка ремонтников

Литературное программирование

Создание литературных скриптов

В VS Code открыл `scripts/mmc_queue_literate.jl` — литературный вариант модели M/M/c. Структура документа: заголовок `# #` Одноканальная и многоканальная СМО M/M/c, секции `# ##` Фиксация генератора случайных чисел, `# ##` Параметры модели, `# ##` Распределения. Параметры те же: `rng = StableRNG(123)`, `num_customers = 1000`, `num_servers = 2`, `mu = 0.5`, `lam = 0.9` (рис. @fig-017).

```

1  # # Одноканальная и многоканальная СМО M/M/c
2  #
3  # Моделирование системы массового обслуживания с экспоненциальными
4  # распределениями интервалов поступления и времени обслуживания.
5
6  using DrWatson
7  @quickactivate "project"
8  using StableRNGs
9  using Distributions
10 using ConcurrentSim
11 using ResumableFunctions
12 using DataFrames
13 using Statistics
14 using CSV
15 using Plots
16
17 # ## Фиксация генератора случайных чисел
18
19 rng = StableRNG(123)
20
21 # ## Параметры модели
22
23 num_customers = 1000
24 num_servers = 2
25
26 mu = 0.5 # интенсивность обслуживания
27 lam = 0.9 # интенсивность поступления заявок
28
29 # ## Распределения
30
31 arrival_dist = Exponential(1 / lam)
32 service_dist = Exponential(1 / mu)
33

```

Figure 21: `mmc_queue_literate.jl`: литературный стиль с секциями и комментариями-заголовками

Открыл `scripts/ross_literate.jl` — литературный вариант

модели Росса. Заголовок: `# # Параметрическое исследование модели Росса`. Секции: `# ## Параметры эксперимента` и `# ## Аналитическая оценка времени до отказа`. Параметры: `RUNS = 10`, `machine_counts = [5, 10, 20, 30]`, `S = 3`, `repairers = 2`, `LAMBDA = 100.0`, `MU = 1.0` ([рис. @fig-018]).

```

1  # # Параметрическое исследование модели Росса
2  #
3  # Анализ влияния числа работающих машин на надёжность системы
4  # и нагрузки ремонтной бригады.
5
6  using DrWatson
7  @quickactivate "project"
8  using ResumableFunctions
9  using ConcurrentSim
10 using Distributions
11 using StableRNGs
12 using DataFrames
13 using Statistics
14 using CSV
15 using Plots
16
17 # ## Параметры эксперимента
18
19 RUNS = 10
20 machine_counts = [5, 10, 20, 30]
21 S = 3
22 repairers = 2
23 LAMBDA = 100.0
24 MU = 1.0
25
26 # ## Аналитическая оценка времени до отказа
27
28 function analytical_estimate(N, S, λ, μ)
29     λ_total = N / λ
30     reserve_factor = S + 1
31     return reserve_factor / max(λ_total - μ, 1e-6)
32 end
33

```

Figure 22: ross_literate.jl: литературный стиль, параметры эксперимента

Запустил оба литературных скрипта. mmc_queue_literate.jl выдал: среднее время ожидания 5.900882801930325, среднее время обслуживания 1.9636150889433126. ross_literate.jl выдал DataFrame 4×4 с теми же значениями, что и DrWatson-версия: $N=5 \rightarrow 7.43 \times 10^5$, $N=10 \rightarrow 29970.5$, $N=20 \rightarrow 2658.15$, $N=30 \rightarrow 476.449$ ([рис. @fig-019]).

```

drwatson@davdaev ~$ cd /2026-1--study--simulation-modeling/labs/lab07/project$ code scripts/mmc_queue_literate.jl
drwatson@davdaev ~$ cd /2026-1--study--simulation-modeling/labs/lab07/project$ code scripts/ross_literate.jl
drwatson@davdaev ~$ cd /2026-1--study--simulation-modeling/labs/lab07/project$ julia --project=. scripts/mmc_queue_literate.jl
Среднее время ожидания: 5.900882801930325
Среднее время обслуживания: 1.9636150889433126
Could not create decoration from factory! Running with no decorations.
Результаты сохранены в data/ и plots/

drwatson@davdaev ~$ cd /2026-1--study--simulation-modeling/labs/lab07/project$ julia --project=. scripts/ross_literate.jl
4x4 DataFrame
  Row  machines  crash_time  analytical  avg_queue
     Int64  Float64  Float64      Float64
1      5      7.43e5      4.8e0      8.47979e-5
2     10     29970.5      4.8e0      8.000717156
3     20      2658.15      4.8e0      8.000723249
4     30      476.449      4.8e0      8.0278657
Could not create decoration from factory! Running with no decorations.
Результаты сохранены в data/ и plots/
drwatson@davdaev ~$ cd /2026-1--study--simulation-modeling/labs/lab07/project$

```

Figure 23: Запуск литературных скриптов mmc и ross: совпадающие результаты с DrWatson-версией

Генерация чистого кода через Literate.jl

В Julia REPL выполнил серию вызовов `Literate.script(...)` для всех четырёх скриптов. Каждый вызов выводил сообщения `[ Info: generating plain script file from ...` и `[ Info: writing result to ....` Сгенерированы файлы: `mmc_queue_clean.jl`, `ross_clean.jl`, `ross_parametrized_clean.jl`, `mmc_parametrized_clean.jl` (рис. @fig-020)).

```

jll@literate:~/scripts/ross_literate.jl:~$ scripts = rm -rf *.clean
jll@literate:~/scripts/ross_literate.jl:~$ Info: generating plan script file from /~/2020-1-study-simulation-modeling/labs/lab07/project/scripts/nmc_queue_literate.jl
jll@literate:~/scripts/ross_literate.jl:~$ Info: writing result to /~/2020-1-study-simulation-modeling/labs/lab07/project/scripts/nmc_queue_clean.jl
~/home/jgavdev/2020-1-study-simulation-modeling/labs/lab07/project/scripts/ross_literate.jl:~$
jll@literate:~/scripts/ross_literate.jl:~$ scripts = rm -rf *.clean
jll@literate:~/scripts/ross_literate.jl:~$ Info: generating plan script file from /~/2020-1-study-simulation-modeling/labs/lab07/project/scripts/ross_literate.jl
jll@literate:~/scripts/ross_literate.jl:~$ Info: writing result to /~/2020-1-study-simulation-modeling/labs/lab07/project/scripts/ross_clean.jl
~/home/jgavdev/2020-1-study-simulation-modeling/labs/lab07/project/scripts/ross_literate.jl:~$
jll@literate:~/scripts/ross_parametrized_literate.jl:~$ scripts = rm -rf *.clean
jll@literate:~/scripts/ross_parametrized_literate.jl:~$ Info: generating plan script file from /~/2020-1-study-simulation-modeling/labs/lab07/project/scripts/ross_parametrized_literate.jl
jll@literate:~/scripts/ross_parametrized_literate.jl:~$ Info: writing result to /~/2020-1-study-simulation-modeling/labs/lab07/project/scripts/ross_parametrized_clean.jl
~/home/jgavdev/2020-1-study-simulation-modeling/labs/lab07/project/scripts/ross_parametrized_literate.jl:~$

```

Figure 24: Literate.script: генерация чистых .jl скриптов для всех четырёх моделей

При попытке сгенерировать Jupyter Notebook для `ross_parametrized_literate.jl` возникла ошибка `InterruptException` в блоке кода с циклом по `param_sets`. Ошибка произошла при вызове `Literate.notebook(...)` в интерактивной среде с `InterruptException` из `Literate.jl:831` (рис. @fig-021).

[illegible]

Figure 25: Ошибка InterruptException при генерации notebook для ross parametrized literate

Quarto-документация в браузере

Открыл сгенерированные HTML-файлы в браузере. Файл `markdown/mmc_parametrized_literate.html` отображает заголовок «Параметризованные эксперименты: модель М/М/с», секцию «Наборы параметров для экспериментов» с подсвеченным кодом Julia (рис. @fig-026).

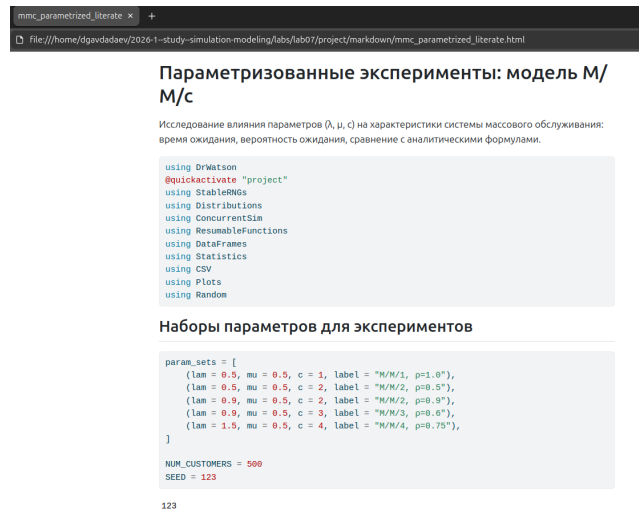


Figure 26: HTML-документация `mmc_parametrized_literate` в браузере

Файл `markdown/mmc_queue_literate.html` — «Одноканальная и многоканальная СМО M/M/c» с секциями «Фиксация генератора случайных чисел» и «Параметры модели». Выведен результат инициализации RNG: `StableRNGs.LehmerRNG(state=0x...00f7)` ([рис. @fig-027]).

Файл `markdown/ross_literate.html` — «Параметрическое исследование модели Росса» с секциями «Параметры моделирования» и «Аналитическая оценка времени до падения» ([рис. @fig-028]).

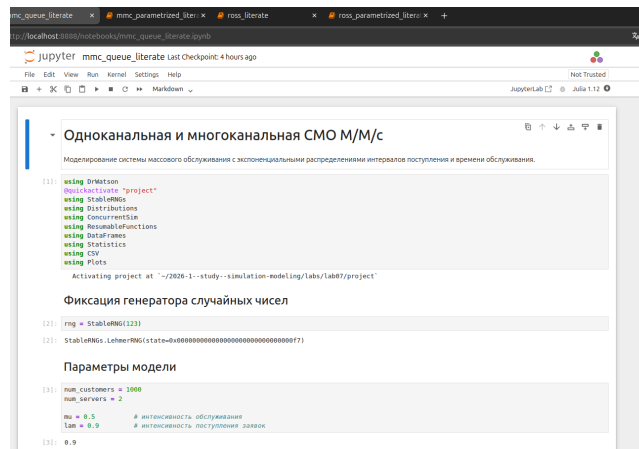
Структура файлов `scripts/`

В файловом менеджере проверил содержимое папки `project/scripts/`: четыре подпапки с литературными скриптами (`mmc_parametrized_literate`, `mmc_queue`, `ross`, `ross_parametrized_literate`) и набор файлов: `intro.jl`, `mmc_parametrized_clean.jl`, `mmc_parametrized_literate.jl`, `mmc_queue_clean.jl`, `mmc_queue_literate.jl`, `ross_clean.jl`, `ross_literate.jl`, `ross_parametrized_clean.jl`, `ross_parametrized_literate.jl` ([рис. @fig-029]).

Jupyter Notebooks

Открыл Jupyter по адресу `localhost:8888`. Notebook `mmc_queue_literate.ipynb` открыт: первая ячейка с импортами пакетов выполнена, `@quickactivate "project"` выдал «Activating project at ~/2026-1-study-

simulation-modeling/labs/lab07/project». Вторая ячейка: `rng = StableRNG(123) → StableRNGs.LehmerRNG(state=0x...00f7)`. Третья ячейка: `num_customers = 1000, num_servers = 2, lam = 0.9` ([рис. @fig-030]).



```

[1]: using DiscreteEventSimulation
    using StableRNG
    using Distributions
    using ConcurrentCollections
    using ResumableFunctions
    using DataFrames
    using Statistics
    using CSV
    using Plots

    Activating project at "/2020-1--study--simulation-modeling/labs/lab07/project"

Фиксация генератора случайных чисел

[2]: rng = StableRNG(123)
[2]: StableRNGs.LehmerRNG(state=0x000000000000000000000000000000f7)

Параметры модели

[3]: num_customers = 1000
    num_servers = 2
    mu = 0.5 # интенсивность обслуживания
    lam = 0.9 # интенсивность поступления заявок

[3]: 0.9
  
```

Figure 30: Jupyter Notebook `mmc_queue_literate.ipynb`: выполнение ячеек

Notebook `mmc_parametrized_literate.ipynb`: заголовок «Параметризованные эксперименты: модель М/М/с», ячейка с `param_sets` выполнена, результат ячейки `SEED = 123` ([рис. @fig-031]).

Notebook `ross_literate.ipynb`: «Параметрическое исследование модели Росса», секции «Параметры эксперимента» и «Аналитическая оценка времени до отказа». Ячейка с параметрами: `RUNS = 10, machine_counts = [5, 10, 20, 30], MU = 1.0` → результат 1.0 ([рис. @fig-032]).

Notebook `ross_parametrized_literate.ipynb`: «Параметризованные эксперименты: модель Росса», ячейка с `param_sets` из 7 конфигураций выполнена, `RUNS = 30, SEED = 42` ([рис. @fig-033]).

Выводы

Реализован дискретно-событийный подход к моделированию СМО на Julia: модель М/М/2 с параметрами $\lambda = 0.9, \mu = 0.5$ показала среднее время ожидания 5.90 единиц; гистограмма ожидания имеет пик у нуля (≈ 260 заявок обслуживаются без очереди) и хвост до 28 единиц ([рис. @fig-022]).

```

[1]: using OrMatson
    @instantiate "project"
    using StabPMRGs
    using Distributions
    using ConcurrentSin
    using PersonalIntections
    using DataFrames
    using Statistics
    using CSV
    using Plots
    using Random

Наборы параметров для экспериментов

[2]: param_sets = [
    (lam = 0.5, mu = 0.5, c = 1, label = "70%/1, p=1.0"),
    (lam = 0.5, mu = 0.5, c = 2, label = "70%/2, p=0.5"),
    (lam = 0.9, mu = 0.5, c = 2, label = "70%/2, p=0.5"),
    (lam = 0.9, mu = 0.5, c = 3, label = "70%/3, p=0.3"),
    (lam = 1.5, mu = 0.5, c = 4, label = "70%/4, p=0.75"),
  ]

  NUM_CUSTOMERS = 500
  SEED = 123

[3]: 123
  
```

Figure 31: Jupyter Notebook mmc_parametrized_literate.ipynb: param_sets с 5 конфигурациями

```

[1]: using OrMatson
    @instantiate "project"
    using PersonalIntections
    using ConcurrentSin
    using Distributions
    using StabPMRGs
    using DataFrames
    using Statistics
    using CSV
    using Plots

Параметры эксперимента

[2]: NUMC = 10
    machine_counts = {5, 10, 20, 30}
    S = 3
    repairers = 2
    LAMBDA = 100.0
    MU = 1.0

[3]: 1.0
  
```

Figure 32: Jupyter Notebook ross_literate.ipynb: параметры и аналитическая оценка

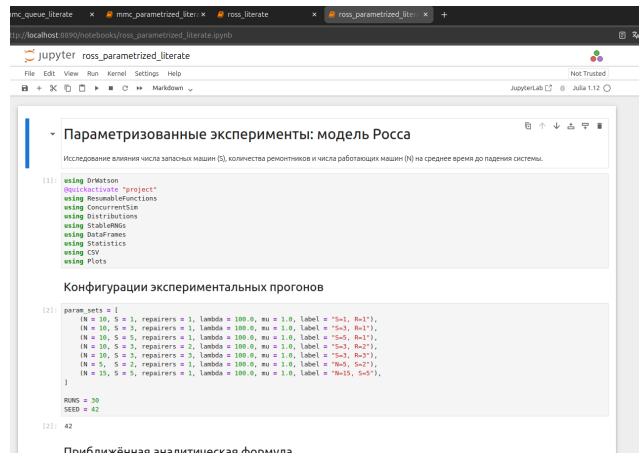


Figure 33: Jupyter Notebook ross_parametrized_literate.ipynb: 7 конфигураций, RUNS=30, SEED=42

Параметрическое исследование М/М/с выявило, что при $\rho = 1.0$ среднее ожидание достигает 19.20 и $P(\text{ожидания}) = 0.926$ — система нестационарна. При увеличении числа каналов до $c = 3$ при той же нагрузке $\rho = 0.6$ среднее ожидание падает до 0.54 и $P(\text{ожидания}) = 0.342$. Симуляционные вероятности ожидания согласуются с формулой Эрланга: точки лежат вдоль диагонали на графике «аналитика vs симуляция» ([рис. @fig-024]).

В модели Росса среднее время до падения резко убывает с ростом N : при $N = 5$ оно составляет 7.43×10^5 , при $N = 30$ — всего 476 часов. Добавление запасных машин эффективнее добавления ремонтников: конфигурация $S = 5, R = 1$ даёт $\text{avg_crash} = 9.20 \times 10^5$, тогда как $S = 3, R = 3$ — только 87119 ([рис. @fig-025]).

Освоены инструменты литературного программирования: из скриптов *_literate.jl через Literate.script() сгенерированы чистые .jl-файлы, а через Quarto получены HTML-документы с подсвеченным кодом. Четыре Jupyter Notebook выполнены и подтвердили воспроизводимость результатов.

```
{{< include ../project/markdown/mmc_queue_literate/mmc_queue_literate.qmd
>}}
```

```
{{< include ../project/markdown/mmc_parametrized_literate/mmc_parametrized_literate.qmd
>}}
```

```
{{< include ../project/markdown/ross_literate/ross_literate.qmd >}}
```

```
{{< include ../project/markdown/ross_parametrized_literate/ross_parametrized_literate.qmd
```

>}}

Список литературы

1. Ross S. M. Simulation. — 5th ed. — Academic Press, 2013. — 312 с.
2. Кулябов Д. С., Королькова А. В. Имитационное моделирование. Практикум. — М.: РУДН, 2024.
3. Kendall D. G. Stochastic Processes Occurring in the Theory of Queues // Annals of Mathematical Statistics. — 1953. — Vol. 24, no. 3. — P. 338-354.
4. Datseris G. et al. Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity // SIMULATION. — 2022. — Vol. 99, no. 1.
5. DrWatson.jl Documentation. — URL: <https://juliadynamics.github.io/DrWatson.jl/stable/>